

Практическая работа по Информационным технологиям
Базовый набор регистров и альтернативные имена в стандарте ABI ассемблера
RISC-V. Арифметические и логические операции R-типа

В первой части этой работы рассмотрен базовый набор регистров и альтернативные имена в стандарте ABI (application binary interface).

Базовое подмножество команд использует следующий набор регистров (таблица 1): специальный регистр `x0 (zero)`, 31 целочисленный регистр общего назначения (`x1 — x31`), регистр счётчика команд (PC, используется только косвенно), а также множество CSR (Control and Status Registers, может быть адресовано до 4096 CSR).

Таблица 1. Набор регистров RISC-V (регистровый файл).

Номер	Название	Описание
x0	zero	Константа нуля (zero register)
x1	ra	Адрес возврата (return address)
x2	sp	Указатель стека (stack pointer)
x3	gp	Глобальный указатель (global pointer)
x4	tp	Указатель потока (thread pointer)
x5-x7	t0-t2	Временные переменные (temporary registers)
x8	s0 / fp	Сохраняемая переменная / Указатель фрейма стека (saved register / frame pointer)
x9	s1	Сохраняемая переменная (saved register)
x10-x11	a0-a1	Аргументы функций/Возвращаемые значения (function arguments / return values)
x12-x17	a2-a7	Аргументы функций (function arguments)
x18-x27	s2-s11	Сохраняемые переменные (saved registers)
x28-x31	t3-t6	Временные переменные (temporary registers)

Для лучшей читаемости кода инструкции ассемблера обычно используют специальные имена, например `s1`, но они также могут использовать номер регистра (например, `x9` для регистра номер 9). В нулевом регистре **x0** всегда хранится константа 0; попытка записать в него другое значение игнорируется. Регистры от **s0** до **s11** (регистры 8–9 и 18–27) и от **t0** до **t6** (регистры 5–7 и 28–31) используются для хранения переменных; **ra** и регистры от **a0** до **a7** служат для вызовов функций. Регистры 2–4 носят имена **sp**, **gp** и **tp**. Нет операций над частями регистров, нет каких-либо выделенных «регистровых пар».

Регистр **gp** (global pointer) представляет собой специальную константу, указывающую в блок данных, где хранятся константы и глобальные переменные. Значение данного регистра инициализируется в начале работы программы (это делает линковщик, когда компирует программу) смещением 0x800 от начала данной секции, что позволяет получать доступ к содержимому секции используя 12-битное смещение со знаком, что значительно

упрощает доступ к константам и глобальным переменным. Изменять значение регистра **gp** самостоятельно не рекомендуется.

Архитектура RISC-V использует только little-endian модель (рис. 1) — первый байт операнда в памяти соответствует наименее значащим битам значений регистрового операнда. RISC-V применяет память с побайтовой адресацией. Это значит, что каждый байт памяти имеет уникальный адрес. Поскольку 32-битное слово состоит из четырех 8-битных байтов, то адрес каждого слова (word address) кратен 4. Старший байт (most significant byte, MSB) находится слева, а младший байт (least significant byte, LSB) — справа.



Рис. 1 Формат little-endian.

Инструкции базового набора имеют длину 32 бита. Формат 32-битной машинной команды (признаки — младшие биты всегда «11» и 2-4 биты ≠ «111»).

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0			
funct7				rs2			rs1		funct3		rd			opcode		R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type		
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode	B-type
imm[31:12]										rd			opcode		U-type		
imm[20]		imm[10:1]				imm[11]		imm[19:12]			rd			opcode		J-type	

Рис. 2 Типы команд и их кодирование для RISC-V

Обозначения на рис. 2:

- rs1 (5бит) — номер регистра, в котором находится первый операнд;
- rs2 (5бит) — номер регистра, в котором находится второй операнд;
- rd (5бит) — номер регистра, в который будет записан результат;
- opcode (7бит) +funct7+funct3 (10бит) определяют операцию;
- imm – непосредственный операнд (константа) в дополнительном коде.

Таким образом, в архитектуре RISC-V используются 6 форматов инструкций. Немного подробнее о каждом типе:

- R-type (регистр),
- I-type (непосредственный операнд ~ константа),
- S-type (хранилище),
- B-type (ветвь),
- U-type (верхний непосредственный)
- J-type (переход)

Арифметические и логические операции R-типа

В 1-й практической работе мы начали знакомиться с арифметическими и логическими операциями I-типа, когда один из операндов был представлен непосредственным значением (I означает Immediate). Теперь освоим похожие арифметические и логические операции, но которые используют два регистра в качестве источника и один регистр в качестве приемника.

Большинство инструкций R-type (таблица 2) являются аналогом операций с непосредственными значениями.

Таблица 2. Операции R-type

Instruction	name	opcode	funct3	description
add	ADD	0110011	0x0	$rd = rs1 + rs2$
sub	SUB	0110011	0x0	$rd = rs1 - rs2$
xor	XOR	0110011	0x4	$rd = rs1 \wedge rs2$
or	OR	0110011	0x6	$rd = rs1 \mid rs2$
and	AND	0110011	0x7	$rd = rs1 \& rs2$
sll	Shift Left Logical	0110011	0x1	$rd = rs1 \ll rs2$
srl	Shift Right Logical	0110011	0x5	$rd = rs1 \gg rs2$
sra	Shift Right Arith	0110011	0x5	$rd = rs1 \ggg rs2$
slt	Set Less Than	0110011	0x2	$rd = (rs1 < rs2) ? 0 : 1$
sltu	Set Less Than Unsigned	0110011	0x3	$rd = (rs1 < rs2) ? 0 : 1$

Примеры:

```
sub  t2, t0, t1    # t2 = t0 - t1
slt  x3, x1, x2    # if x1 < x2 then x3 = 1 else x3 = 0
and  x3, x1, x2    # x3 = x1 & x2
sll  x3, x1, x2    # x3 = x1 << x2
```

Рассмотрим пример более сложных вычислений:

```
a = ((b+3) >> c) - 1
```

Предположим, что a, b, c находятся в регистрах s2, s3 и s4 соответственно. Используем регистр s5 для y0 и регистр s6 для y1:

```
addi s5, s3, 3     # y0 = b + 3
srl  s6, s5, s4     # y1 = y0 >> c
addi s2, s6, -1     # a = y1 - 1
```

Для справки в приложении приведены инструкции I-типа с пояснениями. Также рекомендуется обращаться к источнику [1].

Задание.

Напишите на ассемблере свой вариант программы, которая использует в качестве целых значений переменных x и y последние цифры вашего номера студенческого билета. Например, для номера 431137: $x = 3$, $y = 7$. Результат поместите в регистр $s2$ (x18).

Варианты заданий:

1. $(x + 7) - (y - 3)$
2. $((x + 4) >> 2) + ((y - 1) << 3)$ ($>>$ — логический сдвиг)
3. $((x + 1) << y) - 10$
4. $((x + 8) >> y) + 10$ ($>>$ — арифметический сдвиг)
5. $((x + 1) << 2) - y + 5 >> 1$ ($>>$ — арифметический сдвиг)
6. $x * 6 - y * 3$ (выполнить умножение, используя сдвиг, сложение, вычитание)
7. $2 * (x + 5) - 3 * (y + 1)$
8. $(x + 7) ^ (-2 << 4)$
9. $(x + 8) // 4 + (y + 1) * 2$ (выполнить деление, используя сдвиг, сложение, вычитание)
10. $(x + y) \& (-1 << 5)$
11. $(x - y) | (-1 >> 5)$ ($>>$ — логический сдвиг)
12. Установите y -ий бит x в 1 (номера битов начинаются с 0)
13. Установите y -ий бит x в 0 (номера битов начинаются с 0)
14. $(x + 3) < (y + x) * 2 ? 1 : 0$
15. $(x >> y) - 8$
16. $x * 2 - y * 3$
17. $(y + 6) < (x + 8) ? 1 : 0$
18. $(x >> y) + 10$ ($>>$ — арифметический сдвиг)
19. $(x + 3) - (y - 4)$
20. $(x - y) \& (-1 << 5)$
21. Установите y -ий бит x в 1 (номера битов начинаются с 0)
22. Установите y -ий бит x в 0 (номера битов начинаются с 0)
23. $(3 - x) < (y + 3) ? 1 : 0$
24. $(x - 5) * 2 - y * 3$
25. $(x + 3) * 3 - 4 * y + 3$
26. $(x - 4) | ((y + 8) >> 3)$ ($>>$ — логический сдвиг)
27. $2 * x < (y + 7) ? 1 : 0$
28. $(x + 3) + (y - 5)$
29. $((x - 4) << y) - 10$
30. $(x + 2) < (y + 5) ? 1 : 0$
31. $(x + 3) // 2 + y // 4$
32. $(x < -5) \& (y < 10)$

// - целочисленное деление

^ - xor, | - or, & - and

Вопросы для контроля

1. Что входит в базовый набор инструкций?
2. За что отвечают регистры a2-a7?
3. Поясните, как кодируются инструкции типа R.
4. Как выполнить эти операции без использования инструкции умножения?

$x * 2$

$x * 3$

$x * 4$

$x * 5$

$x * 8$

$x * 31$

5. Как выполнить эти операции без использования инструкции деления?

$x / 2$

$x / 4$

$x / 8$

$x / 32$

Список литературы:

1. [Руководство по набору инструкций RISC-V, том I: Непривилегированный ISA](https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link) URL: https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link
2. RISC-V Ratified Specification URL: <https://riscv.org/specifications/ratified/>
3. RISC-V ISA pages documentation (html) URL: <https://msyksphinz-self.github.io/riscv-isa-doc/html/rvi.html>

Приложение 1

Таблица 3. Операции I-type

Instruction	name	opcode	funct3	description
addi	ADD immediate	0010011	0x0	$rd = rs1 + imm$
xori	XOR immediate	0010011	0x4	$rd = rs1 \wedge imm$
ori	OR immediate	0010011	0x6	$rd = rs1 \vee imm$
andi	AND immediate	0010011	0x7	$rd = rs1 \& imm$
slli	Shift Left Logical Imm.	0010011	0x1	$imm(11:5) = 0x00, rd = rs1 \ll imm(4:0)$
srli	Shift Right Logical Imm.	0010011	0x5	$imm(11:5) = 0x00, rd = rs1 \gg imm(4:0)$
srai	Shift Right Arith Imm.	0010011	0x5	$imm(11:5) = 0x20, rd = rs1 \gg imm(4:0)$
slti	Set Less Than Imm.	0010011	0x2	$rd = (rs1 < imm) ? 0:1$
sltiu	Set Less Than Imm. Unsigned	0010011	0x3	$rd = (rs1 < imm) ? 0:1$

Команды I-type (таблица 3) принимают непосредственное значение *imm* (константу), исходный регистр *rs1* и целевой регистр *rd*.

Поскольку инструкция **addi rd, rs1, imm** использует значение *imm* в качестве значения со знаком, она работает для арифметического сложения ($imm = 0 \dots 2047$) и вычитания ($imm = -2048 \dots -1$), т.е. нет команды *subi*.

Инструкции **xori, ori** и **andi rd, rs1, imm** выполняют битовую операцию *xor, or, and* соответственно. Например, пусть значение регистра *t1* равно 0×9 ; в двоичной системе счисления (обозначается префиксом 0b) это 0b1001. Команда **xori t1, t1, 0x3** выполняет операцию хог над содержимым регистра *t1*, таким образом

$$t1 = t1 \text{ xor } 0 \times 3 = 0b1001 \text{ xor } 0b0011 = 0b1010 = 0xa.$$

Инструкции **slli, srli** и **srai** выполняют операции сдвига, используя непосредственное значение в качестве количества сдвигов. Непосредственным значением является положительное число из 5 бит ($0 \dots 31$).

Логический сдвиг влево выполняется с помощью команды **slli rd, rs1, imm**. Она перемещает влево каждый бит значения исходного регистра *rs1* на число бит, заданное непосредственным значением, и записывает результат в *rd*. При каждом сдвиге младший значащий бит устанавливается равным нулю, а старший значащий бит отбрасывается.

Логический сдвиг вправо выполняется аналогично с помощью **srli rd, rs1, imm**. Регистр *rs1* побитово сдвигается вправо на *immediate*, и результат поступает в *rd*. Здесь самый старший бит заполняется нулем, а младший бит отбрасывается.

Обработка или заполнение старшего значащего бита отличается для

арифметического сдвига вправо, выполняемого с помощью **srai rd, rs1, imm**. Самый старший бит (msb) сохраняется. Если msb равен 0 перед сдвигом, то msb заполняется значением 0 после сдвига. Если msb равен 1 перед сдвигом, то msb заполняется значением 1. Например, пусть значение регистра **t1** равно 0x9, в двоичной системе счисления это 0b1001. Команда **slli t1, t1, 0x2** выполняет логический сдвиг содержимого регистра **t1** влево таким образом

$$t1 = t1 \ll 0x2 = 0b1001 \ll 2 = 0b100100 = 0x24.$$

Оставшиеся две инструкции **slti rd, rs1, imm** и **sltiu rd, rs1, imm** сравнивают содержимое исходного регистра **rs1** с непосредственным значением (**imm**) и устанавливают в **rd** значение 0, если значение исходного регистра **rs1 < imm**. Разница между обеими инструкциями заключается в том, что **slti** использует значения со знаком, и в сравнении используются значения со знаком. Примеры:

slti s2, s6, -1	# if s6<(-1) then s2=1 else s2=0
andi t2, t1, 3	# t3 = t1 & 3
slli t2, t1, 3	# t3 = t1 << 3
addi t2, t1, -3	# t3 = t1 - 3